

Applied Machine Learning — Week 8

PCA · Matrix Factorisation (NNMF) · Data Visualisation (t-SNE)

Comprehensive Revision Notes

Contents

I	Principal Component Analysis (PCA)	2
1	Motivation & Three Derivations	2
2	PCA via Maximum Variance	2
3	PCA via Minimum Reconstruction Error	2
4	PCA via SVD (Practical Computation)	3
5	PCA as a Generative Model	3
6	PCA: Properties & Limitations	3
II	Matrix Factorisation	4
7	Non-Negative Matrix Factorisation (NNMF)	4
7.1	NNMF vs PCA	4
7.2	Applications	4
7.3	NNMF Discussion	4
8	Autoencoders (Nonlinear Extension)	5
III	Data Visualisation (SNE & t-SNE)	6
9	Problem Formulation	6
10	Stochastic Neighbour Embedding (SNE)	6
11	Symmetric SNE	6
12	t-SNE	6
13	Scaling t-SNE	7
IV	Exam-Style Questions	8

Part I

Principal Component Analysis (PCA)

Motivation & Three Derivations

PCA learns a linear projection from $x_i \in \mathbb{R}^D$ to $z_i \in \mathbb{R}^K$ ($K \ll D$) that preserves as much information as possible. There are three equivalent views:

1. **Maximum variance:** find directions of greatest data variation.
2. **Minimum reconstruction error:** find the projection that best reconstructs the original data.
3. **Generative model:** latent Gaussian model with linear decoder.

PCA via Maximum Variance

First Principal Component

Centre the data: $\tilde{x}_i = x_i - \mu$, where $\mu = \frac{1}{N} \sum x_i$. Sample covariance:

$$S = \frac{1}{N} \sum_{i=1}^N \tilde{x}_i \tilde{x}_i^\top = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

The variance of the projection of the data onto a unit vector u is:

$$\text{Var}(\tilde{X}u) = u^\top S u.$$

Maximise subject to $\|u\| = 1$ via Lagrange multipliers:

$$\frac{\partial}{\partial u} [u^\top S u - \lambda(u^\top u - 1)] = 2Su - 2\lambda u = 0 \implies Su = \lambda u.$$

So u is an **eigenvector** of S , and the variance is $u^\top S u = \lambda$ (the eigenvalue). The first PC is the eigenvector with the **largest eigenvalue**.

All K Principal Components

Eigendecompose: $S = V \Lambda V^\top$, where $V = (v_1 | \dots | v_D)$ are eigenvectors ordered by $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D \geq 0$.

Take the top K eigenvectors $V_K = (v_1 | \dots | v_K) \in \mathbb{R}^{D \times K}$.

Variance explained: $\sum_{k=1}^K \lambda_k / \sum_{k=1}^D \lambda_k$. Use a **scree plot** (λ_k vs k) to choose K .

PCA via Minimum Reconstruction Error

Reconstruction View

Approximate each point: $\hat{x}_i = B a_i + \mu$, where $B \in \mathbb{R}^{D \times K}$ is the basis and $a_i \in \mathbb{R}^K$ are the codes.

Minimise reconstruction error: $L = \sum_{i=1}^N \|x_i - B a_i - \mu\|^2$.

Optimal codes: $a_i = B^\top \tilde{x}_i$ (orthogonal projection).

Optimal basis: the top K eigenvectors of S (same as the maximum-variance solution).

Reconstruction error = $\sum_{k>K} \lambda_k$ (sum of discarded eigenvalues).

PCA via SVD (Practical Computation)

SVD Approach

Compute the SVD: $\tilde{X} = UDV^\top$, where $U \in \mathbb{R}^{N \times N}$, D is diagonal (singular values), $V \in \mathbb{R}^{D \times D}$.

The columns of V are the PCs. The singular values satisfy $\sigma_k = \sqrt{N\lambda_k}$.

Projection & reconstruction:

$$z_i = V_K^\top (x_i - \mu), \quad \hat{x}_i = V_K z_i + \mu.$$

Whitening: apply all D PCs $\Rightarrow \text{Cov}(Z) = I$ (uncorrelated, unit-variance features).

PCA as a Generative Model

Probabilistic PCA

$$z \sim \mathcal{N}(0, I_K), \quad x | z \sim \mathcal{N}(Wz + \mu, \sigma^2 I).$$

Marginal: $p(x) = \mathcal{N}(x | \mu, WW^\top + \sigma^2 I)$.

Maximum likelihood solution: $W = V_K(\Lambda_K - \sigma^2 I)^{1/2} R$ for arbitrary rotation R . The columns of W span the same subspace as the top K PCs but are only identified up to rotation.

PCA: Properties & Limitations

Key Properties

- Projections are **orthogonal** and **ordered** by explained variance.
- Defined up to **sign** (or jointly up to rotation).
- **Sensitive to scaling**: changing units of one feature changes its relative variance contribution. Can standardise all features to variance 1, but this has its own issues (may remove informative scale differences).
- **Linear only**: may be inadequate for complex nonlinear structure.
- Doesn't make sense for non-real-valued data.

Applications: dimensionality reduction, visualisation, preprocessing/whitening for downstream algorithms, outlier detection (high reconstruction error), missing data imputation.

Connections:

PCA links **Kernel PCA**: apply PCA in a kernel-induced feature space. **Autoencoders** (nonlinear PCA): replace the linear encoder/decoder with neural networks. **GP-LVM**: Gaussian process latent variable models. **GMM**: also has a latent-variable Gaussian formulation.

Part II

Matrix Factorisation

Non-Negative Matrix Factorisation (NNMF)

NNMF

Factor $X \approx WH$, where $X \in \mathbb{R}^{N \times D}$, $W \in \mathbb{R}_{\geq 0}^{N \times K}$, $H \in \mathbb{R}_{\geq 0}^{K \times D}$.

Each data point: $x_i \approx \sum_{k=1}^K w_{ik} h_k$, where $h_k \in \mathbb{R}_{\geq 0}^D$ is a **basis vector** (“part”) and $w_{ik} \geq 0$ is its weight for point i .

Poisson log-likelihood loss:

$$L(W, H) = \sum_{i,j} [x_{ij} \log(WH)_{ij} - (WH)_{ij}].$$

Multiplicative update rules (Lee & Seung):

$$w_{ik} \leftarrow w_{ik} \cdot \frac{\sum_j h_{kj} x_{ij} / (WH)_{ij}}{\sum_j h_{kj}}, \quad h_{kj} \leftarrow h_{kj} \cdot \frac{\sum_i w_{ik} x_{ij} / (WH)_{ij}}{\sum_i w_{ik}}.$$

Guaranteed to increase (or maintain) the likelihood at each step.

NNMF vs PCA

Key Difference: Additive Parts vs Cancelling Modes

PCA: basis vectors have positive and negative entries; representations involve *cancellation* ($x \approx \mu + \sum z_k v_k$ where z_k can be negative). Eigenfaces are holistic, ghostly patterns.

NNMF: both basis and weights are non-negative, so representation is purely *additive*. Learned parts are localised (eyes, nose, mouth for faces; individual notes for music; individual topics for text). More **interpretable**.

Applications

- **Face decomposition:** NNMF learns face “parts” (eyes, nose, etc.) vs PCA’s holistic eigenfaces.
- **Audio source separation:** decompose a spectrogram to recover individual instruments or notes.
- **Topic modelling** (PLSA): documents as mixtures of topics; words as distributions within topics. TF-IDF preprocessing; related to Latent Dirichlet Allocation (LDA).
- **Recommender systems:** user–item rating matrix; impute missing entries.

NNMF Discussion

Pros: natural for positive data; interpretable additive parts.

Cons: only for non-negative data; non-convex (initialisation-dependent); components not orthogonal or ordered; reducing K can completely change the basis (not a simple subset).

Autoencoders (Nonlinear Extension)

Autoencoder

$$z_i = f_\phi(x_i) \text{ (encoder)}, \quad \hat{x}_i = g_\theta(z_i) \text{ (decoder)}, \quad L(\phi, \theta) = \frac{1}{N} \sum_i \|x_i - g_\theta(f_\phi(x_i))\|^2.$$

When f_ϕ and g_θ are neural networks, this is a **nonlinear** generalisation of PCA. Can capture complex manifold structure. Also possible: kernel PCA, GP-LVM.

Part III

Data Visualisation (SNE & t-SNE)

Problem Formulation

Given $X = \{x_1, \dots, x_N\}$ in high-dimensional space, find low-dimensional embeddings $Y = \{y_1, \dots, y_N\}$ (2D or 3D) that preserve local neighbourhood structure.

Stochastic Neighbour Embedding (SNE)

SNE Transition Probabilities

High-dimensional: probability of “jumping” from x_i to x_j :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0.$$

σ_i is set per-point using a “perplexity” parameter (effective number of neighbours).

Low-dimensional: same form but with fixed variance:

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}, \quad q_{i|i} = 0.$$

Loss: $L(Y) = \sum_i D_{\text{KL}}(p_i \| q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$.
Optimise over the positions Y by gradient descent.

Asymmetry of KL: large cost for placing close-in- x points far apart in y (large p , small q), but *not* vice versa. SNE preserves **local** structure but may lose global structure.

Symmetric SNE

Symmetrise: $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$. Loss: $L_{\text{sym}} = D_{\text{KL}}(p \| q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$.

Gradient: $\frac{\partial L}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)$.

Ensures each data point contributes significantly ($p_i = \sum_j p_{ij} > \frac{1}{2N}$).

t-SNE

t-SNE: The Key Innovation

Replace the Gaussian q with a **Student- t distribution** (1 degree of freedom) in low-D:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}, \quad q_{ii} = 0.$$

Why? The “crowding problem”: when projecting from high-D to 2D, moderately distant points in high-D all get crammed together. The heavy tails of the t -distribution assign non-negligible probability to larger distances in low-D, allowing moderately distant high-D points to spread out. This produces better-separated clusters.

Gradient:

$$\frac{\partial L}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(1 + \|y_i - y_j\|^2)^{-1}(y_i - y_j).$$

△ t-SNE Limitations — Critical!

- **No parametric mapping:** new points cannot be embedded after fitting. Purely for visualisation.
- **Global distances are meaningless:** only *local* neighbourhood structure is preserved. The distance between clusters is **NOT** interpretable.
- **Expensive:** $O(N^2)$ per iteration. Barnes–Hut approximation: $O(N \log N)$.
- **Sensitive to perplexity:** different perplexity values can give very different visualisations.
- **Cannot be used for dimensionality reduction** in a supervised pipeline.

Scaling t-SNE

- **Landmark-based:** select random landmark points, compute transition probabilities via random walks between landmarks, embed only landmarks.
- **Barnes–Hut approximation:** approximate the $O(N^2)$ gradient computation using a tree-based grouping of distant points.
- **Nearest-neighbour graph:** only compute similarities for a set of nearest neighbours (LargeVis).

Part IV

Exam-Style Questions

Exam Q&A:

Q1 — PCA Derivation (4 marks) **Q:** Show that the first principal component is the eigenvector of S with the largest eigenvalue.

A: The variance of the projection onto unit vector u is $u^\top Su$. Maximise subject to $\|u\| = 1$ using a Lagrange multiplier: $\nabla_u[u^\top Su - \lambda(u^\top u - 1)] = 2Su - 2\lambda u = 0$, giving $Su = \lambda u$. So u is an eigenvector and the variance is λ . To maximise variance, pick the eigenvector with the *largest* eigenvalue.

Exam Q&A:

Q2 — PCA vs Random Projection (2 marks, Practice Exam) **Q:** (T/F) PCA is always better than random projection for dimensionality reduction.

A: False. PCA maximises explained variance, but high-variance directions may not be the most discriminative for a downstream task. Random projections (Johnson–Lindenstrauss) approximately preserve all pairwise distances and can perform well for classification even without identifying the “best” directions.

Exam Q&A:

Q3 — NMF vs PCA (3 marks) **Q:** Compare the learned features of PCA and NMF on face images.

A: PCA learns “eigenfaces” — holistic, global patterns with positive and negative components. NMF learns localised *parts* (eyes, nose, mouth) because non-negativity forces additive combination without cancellation. NMF features tend to be more interpretable.

Exam Q&A:

Q4 — t-SNE (3 marks) **Q:** Why does t-SNE use a Student- t distribution instead of a Gaussian in the low-dimensional space?

A: The “crowding problem”: in high-D, there is ample room for moderately-distant points, but in 2D they are forced close together. A Gaussian q drops off quickly, making it expensive to place these points at moderate distances. The heavy-tailed t -distribution assigns non-negligible probability to larger low-D distances, allowing these points to spread out. This produces cleaner cluster separation.

Exam Q&A:

Q5 — t-SNE for Classification (2 marks) **Q:** Can you use t-SNE embeddings as features for a downstream classifier?

A: No. t-SNE has no parametric mapping — it optimises positions for a fixed set of points. New test points cannot be embedded. Use PCA, autoencoders, or random projections for dimensionality reduction in a supervised pipeline.

Exam Q&A:

Q6 — PCA Outlier Detection (2 marks) **Q:** Explain how PCA can be used for outlier detection.

A: Project each point to the top- K PCA subspace and reconstruct. Points with high reconstruction error $\|x_i - \hat{x}_i\|^2$ do not fit well in the principal subspace and are likely outliers or anomalous.

Fei Gao